



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Notebook 03: Data Warehouse

Objetivos:

1. Implementar un data mart (data warehouse) en postgres
2. Cargar datos de la base de datos transaccional db_sports a las tablas del data mart
3. Consultar el data mart

Artefactos creados:

- PostgreSQL: tablas de data warehouse

Copiar archivos .sql

Shell

```
# Descargar y Descomprimir archivo .zip con archivo .sql

# Copiar archivo .sql a /var/lib/postgresql
sudo cp <ruta>/dwh_sports/dwh_sports.sql /var/lib/postgresql/
```

Asegúrese de estar en directorio project y activar ambiente de Python

Shell

```
# Ingresar a postgres
sudo -u postgres psql
```

Dentro del prompt **postgres=#**, ejecutar:

SQL

```
-- Crear base de datos del electiva
CREATE DATABASE dwh_sports;

-- Conectar a la base de datos db_sports
\c dwh_sports

-- Crear tablas
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
\i /var/lib/postgresql/dwh_sports.sql
```

```
-- Salir
```

```
\q
```

Iniciar sesión de PySpark - Conexión a base de datos

Python

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

```
spark = SparkSession.builder \
    .appName("Initial Load DWH") \
    .config("spark.jars", "/usr/local/spark-3.5.8/jars/postgresql-42.7.3.jar") \
    .getOrCreate()
```

```
dwh_jdbc_url = "jdbc:postgresql://localhost:5432/dwh_sports"
```

```
db_jdbc_url = "jdbc:postgresql://localhost:5432/db_sports"
```

```
db_properties = {
    "user": "postgres",
    "password": "abc123",
    "driver": "org.postgresql.Driver"
}
```

Poblar la dimensión: dim_time

Python

```
# 1. Generar secuencia de 1000 días a partir del 2018-01-01
```

```
df_dates = spark.sql("""
```

```
    SELECT EXPLODE(SEQUENCE(DATE '2018-01-01', DATE '2018-01-01' + INTERVAL 1000
DAY, INTERVAL 1 DAY)) as fecha
    """)
```

2. Transformar atributos y generar llave subrogada 'dateid' (incremental)

```
window_time = Window.orderBy("fecha")

df_dim_time = df_dates.select(
    F.row_number().over(window_time).cast("integer").alias("dateid"),
    F.year("fecha").cast("integer").alias("year"),
    F.month("fecha").cast("smallint").alias("month"),
    F.dayofmonth("fecha").cast("smallint").alias("day"),
    F.dayofweek("fecha").cast("smallint").alias("week_day"),
    F.date_format("fecha", "E").cast("string").alias("week_day_desc"),
    F.date_format("fecha", "MMMM").cast("string").alias("month_desc")
)
```

```
df_dim_time.show()
```

3. Carga inicial a PostgreSQL

```
df_dim_time.write \
    .jdbc(url=dwh_jdbc_url, table="dim_time", mode="append",
    properties=db_properties)

print("Dimensión dim_time cargada exitosamente.")
```

Consultar la tabla: dim_time en la base de datos: dwh_sports

Shell

```
# Ingresar a postgres
```

```
sudo -u postgres psql
```

```
# Ingresar a postgres
```

```
\c dwh_sports
```

```
# Consultar la tabla: dim_time
```

```
SELECT * FROM dim_time LIMIT 5;
```

```
dateid | year | month | day | week_day | week_day_desc | month_desc
```

```
-----+-----+-----+-----+-----+-----+-----
      1 | 2018 |      1 |    1 |         2 | Mon          | January
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
2 | 2018 | 1 | 2 | 3 | Tue | January
3 | 2018 | 1 | 3 | 4 | Wed | January
4 | 2018 | 1 | 4 | 5 | Thu | January
5 | 2018 | 1 | 5 | 6 | Fri | January
(5 rows)
```

```
# Consultar la cantidad total de registros en la tabla: dim_time
SELECT COUNT(*) FROM dim_time;

count
-----
1001

# Salir
\q
```

Poblar la dimensión: dim_bodyparts

```
Python

# Consultar datos operacionales de lesiones
df_injury_phases = spark.read.jdbc(url=db_jdbc_url, table="injury_phases",
properties=db_properties)

# Obtener tipos únicos de partes del cuerpo / lesiones
df_bodyparts = df_injury_phases.select("injury_type").distinct() \
    .filter(F.col("injury_type").isNotNull())

# Generar llave subrogada incremental usando row_number()
window_spec = Window.orderBy("injury_type")
df_dim_bodyparts = df_bodyparts.withColumn("id", F.row_number().over(window_spec)) \
    .select(
        F.col("id").cast("integer"),
        F.col("injury_type").cast("string").alias("bodypart")
    )
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
)  
  
df_dim_bodyparts.show()  
  
# Guardar en PostgreSQL  
df_dim_bodyparts.write.jdbc(url=dwh_jdbc_url, table="dim_bodyparts",  
mode="append", properties=db_properties)  
  
print("Dimensión dim_bodyparts cargada exitosamente.")
```

Consultar la tabla: dim_bodyparts en la base de datos: dwh_sports

```
Shell  
# Ingresar a postgres  
sudo -u postgres psql  
  
# Ingresar a postgres  
\c dwh_sports  
  
# Consultar la tabla: dim_time  
SELECT * FROM dim_bodyparts;
```

```
id | bodypart  
---+-----  
0 | stomach  
1 | fibula  
2 | trapezius  
3 | abdomen  
4 | chest  
5 | other-excused  
6 | hand  
...  
  
# Salir  
\q
```

Poblar la dimensión: dim_persons

Python

```
# Consultar tablas operacionales de personas y nombres
df_persons = spark.read.jdbc(url=db_jdbc_url, table="persons",
                             properties=db_properties)
df_display_names = spark.read.jdbc(url=db_jdbc_url, table="display_names",
                                    properties=db_properties)

df_persons.show()
df_display_names.show()

# Cruzar y manejar nulos con coalesce
df_persons_joined = df_persons.join(df_display_names, df_persons["id"] ==
                                     df_display_names["id"], "left") \
    .select(
        df_persons["id"].alias("entity_id"),
        F.coalesce(F.col("first_name"), F.lit("N/A")).alias("first_name"),
        F.coalesce(F.col("last_name"), F.lit("N/A")).alias("last_name")
    ).distinct()

df_persons_joined.show()

# Generar llave subrogada (id) y atributos de control
window_person = Window.orderBy("entity_id")
df_dim_persons = df_persons_joined.withColumn("id",
        F.row_number().over(window_person)) \
    .select(
        F.col("id").cast("integer"),
        F.col("first_name").cast("string"),
        F.col("last_name").cast("string"),
        F.col("entity_id").cast("integer"),
        F.lit(1).cast("integer").alias("version"),
        F.current_timestamp().alias("date_from"),
        F.lit(None).cast("timestamp").alias("date_to")
    )
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
df_dim_persons.show()

# Guardar en PostgreSQL
df_dim_persons.write.jdbc(url=dwh_jdbc_url, table="dim_persons", mode="append",
properties=db_properties)

print("Dimensión dim_persons cargada exitosamente.")
```

Consultar la tabla: dim_bodyparts en la base de datos: dwh_sports

```
Shell

# Ingresar a postgres
sudo -u postgres psql

# Ingresar a postgres
\c dwh_sports

# Consultar la tabla: dim_time
SELECT * FROM dim_persons;
```

id	first_name	last_name	entity_id	version	date_from	date_to
1	N/A	N/A	1	1	2026-09-23 23:23:40.238348	
2	N/A	N/A	2	1	2026-09-23 23:23:40.238348	
3	Chicago	White Sox	3	1	2026-09-23 23:23:40.238348	
4	Cleveland	Indians	4	1	2026-09-23 23:23:40.238348	
5	Detroit	Tigers	5	1	2026-09-23 23:23:40.238348	

```
(5 rows)

# Salir
\q
```

Poblar la dimensión: ft_injuries



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Python

```
# Leer dimensiones ya almacenadas en el DW para recuperar las llaves sustitutas
dw_dim_bodyparts = spark.read.jdbc(url=dwh_jdbc_url, table="dim_bodyparts",
properties=db_properties)
dw_dim_persons = spark.read.jdbc(url=dwh_jdbc_url, table="dim_persons",
properties=db_properties)

# Filtrar eventos posteriores al 2007-07-31 desde la fuente operacional
df_injuries_src = spark.read.jdbc(url=db_jdbc_url, table="injury_phases",
properties=db_properties) \
    .filter(F.col("start_date_time") > "2007-07-31")

# Normalizar las cadenas de texto en la fuente operacional y en la dimensión para
evitar fallos por espacios o mayúsculas
df_injuries_cleaned = df_injuries_src \
    .withColumn("clean_injury_type", F.lower(F.trim(F.col("injury_type"))))

dw_dim_bodyparts_cleaned = dw_dim_bodyparts \
    .withColumn("clean_bodypart", F.lower(F.trim(F.col("bodypart"))))

# Resolver llaves foráneas haciendo join con las dimensiones
df_facts = df_injuries_cleaned \
    .join(dw_dim_bodyparts_cleaned, df_injuries_cleaned["clean_injury_type"] ==
dw_dim_bodyparts_cleaned["clean_bodypart"], "inner") \
    .join(dw_dim_persons, df_injuries_cleaned["person_id"] ==
dw_dim_persons["entity_id"], "inner")

# Seleccionar y estructurar la tabla de hechos con su métrica de cantidad
df_ft_injuries = df_facts.select(
    F.date_format(F.col("start_date_time"),
"yyyyMMdd").cast("integer").alias("date"),
    dw_dim_bodyparts["id"].alias("id_bodyparts"), # id de dim_bodyparts
    dw_dim_persons["id"].alias("id_person"), # id de dim_persons
    F.lit(1).cast("integer").alias("quantity") # Métrica de conteo
)

df_ft_injuries.show()
```




PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
# Guardar en PostgreSQL
df_ft_injuries.write.jdbc(url=dwh_jdbc_url, table="ft_injuries", mode="overwrite",
properties=db_properties)

print("Tabla de Hecho ft_injuries cargada exitosamente.")
```

Consultar la tabla: dim_bodyparts en la base de datos: dwh_sports

Shell

```
# Ingresar a postgres
```

```
sudo -u postgres psql
```

```
# Ingresar a postgres
```

```
\c dwh_sports
```

```
# Consultar la tabla: dim_time
```

```
SELECT * FROM ft_injuries;
```

date	id_bodyparts	id_person	quantity
20071102	6	2366	1
20070803	25	148	1
20070910	33	2563	1
20070803	33	243	1
20071024	27	2580	1
20071104	36	2387	1
20070803	33	516	1
20070814	6	1650	1
20071029	47	2572	1
20071009	10	2393	1

```
# Salir
```

```
\q
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Consulta 1: Esta consulta se enfoca exclusivamente en la relación entre los hechos de lesiones y la dimensión de partes del cuerpo, agrupando y sumando las cantidades.

Une la tabla de hechos de lesiones (ft_injuries) con la dimensión de partes del cuerpo (dim_bodyparts), sumando la cantidad total de incidentes y ordenándolos de mayor a menor para obtener el top.

```
SQL
SELECT
    b.bodypart,
    SUM(f.quantity) AS total_injuries
FROM ft_injuries f
INNER JOIN dim_bodyparts b ON f.id_bodyparts = b.id
GROUP BY b.bodypart
ORDER BY total_injuries DESC
LIMIT 10;
```

Resultado consulta:

```
Shell
bodypart | total_injuries
-----+-----
knee     |          144
ankle    |           60
shoulder |           54
other-excused |          41
hamstring |           31
back     |           29
groin    |           25
foot     |           24
undisclosed |          22
neck     |           21
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Consulta 2: Esta consulta cruza la tabla de hechos con la dimensión de personas (dim_persons), permitiendo identificar quiénes concentran la mayor cantidad de incidentes reportados.

Relaciona los eventos de la tabla de hechos con la dimensión de personas/entidades (dim_persons), agrupando por nombre y apellido para identificar cuáles concentran la mayor cantidad de reportes.

SQL

```
SELECT
  p.first_name,
  p.last_name,
  SUM(f.quantity) AS total_injuries
FROM ft_injuries f
INNER JOIN dim_persons p ON f.id_person = p.id
GROUP BY p.first_name, p.last_name
ORDER BY total_injuries DESC
LIMIT 10;
```

Resultado consulta:

Shell

first_name	last_name	total_injuries
N/A	N/A	131
Marcus	Gwyn	1
Fred	Wakefield	1
Nicklas	Backstrom	1
Brian	Barden	1
Brian	Rogers	1
Ruddy	Lugo	1
Mathieu	Dandenault	1
Adam	Loewen	1
Erick	Dampier	1



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Consulta 3: Agrupación de lesiones por Año y Mes. Dado que la columna date en ft_injuries es un entero con formato YYYYMMDD (ej. 20070804), es posible extraer el año y el mes directamente mediante manipulación de texto o cadenas

Toma la fecha almacenada como un número entero (YYYYMMDD) en la tabla de hechos, la convierte temporalmente a texto para extraer el año y el mes mediante subcadenas, y agrupa los datos cronológicamente.

SQL

```
SELECT
    SUBSTRING(CAST(f.date AS VARCHAR), 1, 4) AS injury_year,
    SUBSTRING(CAST(f.date AS VARCHAR), 5, 2) AS injury_month,
    SUM(f.quantity) AS total_injuries
FROM ft_injuries f
GROUP BY
    SUBSTRING(CAST(f.date AS VARCHAR), 1, 4),
    SUBSTRING(CAST(f.date AS VARCHAR), 5, 2)
ORDER BY injury_year ASC, injury_month ASC;
```

Resultado consulta:

Shell

injury_year	injury_month	total_injuries
2007	08	124
2007	09	75
2007	10	131
2007	11	276
2008	08	1



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Consulta 4: Obtener un listado de las 15 combinaciones con mayor impacto y frecuencia de lesiones, mostrando el nombre y apellido de la persona afectada, la parte del cuerpo lesionada, el volumen total acumulado de incidentes y la cantidad de eventos independientes registrados. Los resultados se agrupan detalladamente por cada persona y zona anatómica, ordenándose de forma descendente primero por el total de lesiones acumuladas y luego por la cantidad de eventos ocurridos.

SQL

```
SELECT
    p.first_name,
    p.last_name,
    b.bodypart,
    SUM(f.quantity) AS total_injuries,
    COUNT(f.date) AS incident_events
FROM ft_injuries f
INNER JOIN dim_persons p ON f.id_person = p.id
INNER JOIN dim_bodyparts b ON f.id_bodyparts = b.id
GROUP BY
    p.first_name,
    p.last_name,
    b.bodypart
ORDER BY
    total_injuries DESC,
    incident_events DESC
LIMIT 15;
```

Resultado consulta:

Shell

first_name	last_name	bodypart	total_injuries	incident_events
N/A	N/A	knee	33	33
N/A	N/A	ankle	13	13
N/A	N/A	undisclosed	10	10
N/A	N/A	foot	9	9
N/A	N/A	other-excused	9	9
N/A	N/A	groin	9	9
N/A	N/A	shoulder	8	8
N/A	N/A	back	6	6
N/A	N/A	hamstring	5	5
N/A	N/A	neck	3	3



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

N/A	N/A	elbow	3	3
N/A	N/A	quadriceps	3	3
N/A	N/A	wrist	2	2
N/A	N/A	calf	2	2
N/A	N/A	achilles-tendon	2	2